

Perimetra

Documentazione completa della Security Report Dashboard

Repository ufficiale

github.com/Emilio-Moscatiello/Perimetra-brief

Versione della documentazione: 15 luglio 2026

Analisi eseguita sul codice sorgente del repository locale

Abstract

Perimetra è una Single Page Application dimostrativa che trasforma un report JSON relativo alla sicurezza di un dominio in una dashboard navigabile. L'applicazione riunisce rischio complessivo, vulnerabilità, data leak, superficie di rete, certificati TLS, sicurezza della posta elettronica e domini potenzialmente simili in un'interfaccia coerente, responsive e consultabile anche da utenti non specialisti. Il progetto adotta React e TypeScript per il frontend, Python per la logica dei dati, FastAPI per il server di sviluppo locale e funzioni serverless Python per la pubblicazione su Vercel.

La natura della demo è un elemento sostanziale della sua interpretazione. Perimetra non effettua scansioni, non interroga servizi di threat intelligence e non monitora infrastrutture reali. Il report aggregato di [cybersonar.demo](#) proviene dal file JSON fornito con il progetto; gli altri due profili, le serie storiche e le righe di dettaglio sono generati in modo pseudo-casuale ma deterministico. Questa relazione documenta l'architettura e il funzionamento effettivi, ricostruiti direttamente dal codice, e separa in modo esplicito dati originari, trasformazioni derivate, contenuti sintetici e valori predefiniti.

Contenuto della relazione

La trattazione procede dall'identità e dagli obiettivi di Perimetra all'esperienza d'uso, per poi descrivere architettura, frontend, backend Python, API, tecnologie, dipendenze e modello dei dati. Seguono un'analisi puntuale della provenienza delle informazioni, le procedure di installazione e distribuzione, i limiti tecnici e di sicurezza, le ipotesi di evoluzione e le conclusioni con il riferimento ufficiale al repository.

Metodo di analisi

La documentazione è stata prodotta leggendo i file sorgente e di configurazione presenti nel repository, fra cui [package.json](#), [package-lock.json](#), [requirements.txt](#), [vercel.json](#), il dataset [summary_seed.json](#), i moduli Python, i contratti TypeScript, le pagine React, i componenti, gli stili e il README. Le affermazioni sul comportamento non derivano quindi soltanto dalla descrizione del progetto, ma sono state confrontate con le funzioni effettivamente invocate. La build di produzione del frontend è stata inoltre verificata con esito positivo tramite `npm run build`.

Identità, contesto e obiettivi

Il problema affrontato

Un report di sicurezza espresso come JSON è adatto allo scambio fra sistemi, ma non costituisce di per sé uno strumento efficace di consultazione. Un lettore deve conoscere la struttura dei campi, sommare conteggi, interpretare punteggi e collegare informazioni sparse fra vulnerabilità, servizi esposti, certificati e data leak. Perimetra affronta questo problema di presentazione: prende un riepilogo strutturato e lo trasforma in una lettura visuale organizzata per temi, con indicatori sintetici, grafici e viste di dettaglio.

L'applicazione ha quindi un obiettivo di visualizzazione e simulazione full stack, non di acquisizione dei dati. Il brief originario richiedeva una pagina HTML capace di elaborare il JSON allegato, privilegiando TypeScript con React e, se utile, Python per il backend. Sugeriva inoltre grafici interattivi, filtri, sezioni di dettaglio e una simulazione API. Perimetra realizza questi aspetti attraverso sei pagine coordinate, un client HTTP tipizzato, due adattatori backend e una sorgente dati condivisa.

Destinatari e valore dimostrativo

La dashboard può essere letta a più livelli. Un responsabile IT ottiene immediatamente il punteggio complessivo e le aree di maggiore esposizione; un analista può filtrare vulnerabilità e leak; un revisore tecnico può osservare la separazione fra interfaccia, API e logica dei dati. L'esperienza simula quella di un prodotto di exposure monitoring, ma i risultati non devono essere usati per decisioni operative: i domini terminano in `.demo`, molte evidenze sono inventate e nessuna attività di scansione è implementata.

Funzionalità realmente implementate

Perimetra consente di scegliere fra tre domini dimostrativi e aggiorna il report associato senza ricaricare l'intera pagina. La panoramica presenta il rischio, gli asset, gli indirizzi IP, i certificati, i conteggi delle vulnerabilità e dei leak, una serie temporale simulata e un riepilogo esecutivo bilingue. Le viste specialistiche espongono vulnerabilità, data leak, porte di rete, WAF, CDN, certificati, configurazione DMARC, blacklist e domini simili. I filtri testuali e categoriali operano nel browser sui dati già ricevuti. Il tema chiaro o scuro e il dominio selezionato persistono nel `localStorage`.

Esperienza d'uso e funzionamento dell'applicazione

Avvio e caricamento iniziale

Il documento HTML di ingresso contiene l'elemento `#root`; il modulo `main.tsx` vi monta React mediante `ReactDOM.createRoot`. L'applicazione è racchiusa in `React.StrictMode`, nel `BrowserRouter` e nell'`AppProvider`. Al primo caricamento il provider determina il tema: usa la preferenza salvata nella chiave `cybersonar-theme`, oppure consulta `prefers-color-scheme`. In modo analogo recupera dalla chiave `cybersonar-domain` l'ultimo dominio selezionato, adottando `cybersonar.demo` come valore predefinito.

Il frontend chiama dapprima `GET /api/domains` per popolare il selettore. In parallelo con il ciclo applicativo, richiede il riepilogo del dominio attivo tramite `GET /api/summary`. Il Context conserva report, caricamento, errore, tema e selezione, rendendoli accessibili alle pagine. Quando l'utente cambia dominio, il valore viene memorizzato e le risorse dipendenti sono richieste nuovamente.

Navigazione e panoramica

La struttura visiva mantiene una sidebar e una barra superiore condivise. La sidebar espone le route `/`, `/vulnerabilita`, `/data-leak`, `/rete`, `/certificati-email` e `/domini-simili`. La barra superiore contiene il selettore del dominio, l'etichetta di rischio, la data di aggiornamento e il controllo del tema. `BrowserRouter` rende la navigazione client-side: il contenuto cambia senza un caricamento completo del documento.

La pagina Panoramica combina il gauge del rischio con i KPI principali. Somma i conteggi per ricavare il totale delle vulnerabilità e dei leak, visualizza le distribuzioni per gravità e categoria, presenta nove punteggi specifici e carica novanta punti di cronologia. Il riepilogo testuale può essere alternato fra italiano e inglese; la pagina rimuove i marcatori Markdown `**` prima della resa e preserva le interruzioni di riga.

Viste di dettaglio

La pagina Vulnerabilità richiede l'elenco generato dal backend, ne calcola la distribuzione per gravità e permette di filtrare per gravità, stato attivo o passivo e corrispondenza testuale su titolo, asset o identificativo. La pagina Data Leak applica un meccanismo analogo alle categorie e allo stato di risoluzione, aggiungendo la ricerca su identificativo e fonte. È importante osservare che il backend limita il campione di leak a venticinque righe per categoria; il totale dichiarato nella testata resta invece quello aggregato del report.

La sezione Rete e Porte ordina le porte in base al numero di esposizioni, associa alle porte note un nome di servizio e presenta asset, indirizzi IPv4 e IPv6, WAF e CDN. Certificati e Sicurezza Email confronta certificati attivi e scaduti, mostra le blacklist, interpreta la stringa `spooftable` e mette in

evidenza la politica DMARC. La sezione Domini simili elenca varianti generate con percentuale di similarità, ipotesi di registrazione e segnale di rischio, offrendo una ricerca sul nome.

Stati di interfaccia e adattamento

L'hook generico `useResource` uniforma caricamento, risultato ed errore delle risorse specialistiche e annulla logicamente gli aggiornamenti di stato se il componente viene smontato. I componenti `LoadingCard` ed `ErrorState` rendono rispettivamente skeleton animati e messaggi accessibili. Il layout passa da sidebar verticale a navigazione orizzontale su schermi più stretti; le griglie si riducono progressivamente a due o una colonna. La preferenza `prefers-reduced-motion` riduce animazioni e transizioni.

Architettura del progetto e flusso dei dati

Separazione dei livelli

Il repository è organizzato in quattro aree funzionali. `frontend` contiene la Single Page Application; `api` raccoglie la logica condivisa e gli handler serverless; `server` offre il server FastAPI locale; `data` conserva il seed JSON e il brief originario. La configurazione `vercel.json` collega queste parti nel deployment. Non esistono database, code di messaggi o servizi esterni: i report vengono costruiti in memoria durante l'importazione di `api/_data.py`.

Il percorso dei dati inizia da `data/summary_seed.json`. Il modulo `_data.py` carica una volta il primo oggetto di `results`, definisce tre profili demo e costruisce il dizionario globale `_REPORTS`. Per `cybersonar.demo` effettua una copia superficiale del report originale e aggiunge l'etichetta "Critico"; per `northwind-retail.demo` e `aurora-fintech.demo` invoca la sintesi deterministica. Le funzioni di accesso restituiscono riepilogo, elenco domini, cronologia e dettagli.

In sviluppo FastAPI importa tale modulo e racchiude i risultati in risposte HTTP validate. In produzione ogni file pubblico della cartella `api` definisce una classe `handler` derivata da `JSONHandler`. Entrambe le modalità usano quindi la stessa logica dati e differiscono soltanto nell'adattatore HTTP. Il frontend usa sempre URL relative sotto `/api`: Vite le inoltra a `127.0.0.1:8000` in sviluppo, mentre Vercel le instrada alle funzioni Python in produzione.

Stato globale e richieste specialistiche

`AppContext` è il punto di coordinamento del frontend. L'elenco dei domini e il riepilogo corrente sono globali perché servono alla barra superiore e a più pagine. Cronologia, vulnerabilità, leak e domini simili sono invece caricati dalle rispettive viste tramite `useResource`. Questa scelta limita le richieste ai dati effettivamente necessari alla pagina corrente. I filtri non modificano il backend: `useMemo` produce sottoinsiemi locali a partire dagli array ricevuti.

Contratti fra frontend e backend

Il file `types/report.ts` descrive in TypeScript la struttura del report e delle risposte specialistiche. Include unioni letterali per severità, categorie di leak e stati, oltre alle interfacce per email, WAF, CDN e serie storica. Questi tipi migliorano il controllo in compilazione ma non validano i JSON a runtime. Il client `api/client.ts` usa un helper generico basato su `fetch`; in caso di stato HTTP non riuscito tenta di leggere `message`, altrimenti genera un errore con il codice di risposta.

Conseguenza architetturale. Se un backend restituisse una struttura diversa da quella dichiarata, TypeScript non la bloccherebbe durante l'esecuzione. Un'evoluzione destinata alla produzione dovrebbe aggiungere validazione runtime, per esempio mediante schemi condivisi o una libreria dedicata.

Frontend React e TypeScript

Bootstrap, routing e componenti strutturali

`main.tsx` inizializza il rendering e importa gli stili globali. `App.tsx` compone `Sidebar`, `TopBar` e `Routes`. La navigazione usa `NavLink` per evidenziare la route attiva e `Link` per i rimandi interni della panoramica. Non è presente un componente per una route sconosciuta: grazie al rewrite di Vercel il documento SPA viene caricato, ma nessuna delle route dichiarate corrisponde e l'area di contenuto rimane priva di pagina.

`TopBar` legge il Context, mostra i domini ottenuti dall'API e converte il punteggio in una classe di stato. `Sidebar` definisce le sei destinazioni e include un marchio SVG creato nel codice. `AppProvider` gestisce gli effetti di caricamento e persistenza; la funzione `refresh` è esposta dal Context e incrementa un contatore capace di ricaricare il report, ma nello stato analizzato non esiste un pulsante o un componente che la invochi.

Componenti di visualizzazione

Il progetto non dipende da librerie esterne di charting o da framework di componenti. `RiskGauge` costruisce un arco SVG usando circonferenza, frazione di tre quarti e punteggio su cento. `TrendLineChart` traduce i punti della cronologia in un percorso SVG, disegna area, griglia e tooltip e usa `ResizeObserver` per adeguarsi al contenitore. `HorizontalBarChart` calcola la larghezza percentuale rispetto al massimo della serie, mentre `StackedBar` normalizza ogni segmento rispetto alla somma complessiva.

`Meter` limita visivamente il valore fra zero e cento, associa il colore alle soglie e pubblica attributi ARIA da misuratore. `StatTile` rende KPI riutilizzabili. `DataTable` è un componente generico: riceve definizioni di colonna, righe e funzione di chiave, rendendo le tabelle di vulnerabilità, leak e domini simili senza duplicare la struttura. Le visualizzazioni sono interattive al passaggio del puntatore, ma non implementano zoom, esportazione o ordinamento manuale delle colonne.

Colori, soglie e formattazione

`lib/colors.ts` centralizza la semantica cromatica. Un punteggio almeno pari a 75 è "Critico"; da 50 a 74 è "Elevato"; da 25 a 49 è "Attenzione"; sotto 25 è "Solido". Le severità sono ricondotte a questi stati: critical a critico, high a elevato, medium ad attenzione, low e info a solido. Le categorie dei leak ricevono colori di serie; `Intl.NumberFormat` con locale italiano produce conteggi compatti quali "838" o, per numeri più grandi, forme abbreviate.

I token CSS definiscono superfici, testi, bordi, otto colori categoriali, una scala sequenziale e quattro stati. Il tema scuro può essere attivato dal sistema o dall'attributo `data-theme`; quello chiaro ha valori espliciti equivalenti. `global.css` realizza una veste da dossier tecnico, con griglia di sfondo, sidebar scura e accento ciano. Sono presenti focus visibili, etichette ARIA in alcuni grafici, navigazione etichettata e riduzione del movimento. Non risulta tuttavia eseguito o incluso un audit WCAG, né sono presenti test con screen reader.

Le sei pagine

`Overview.tsx` è la vista più composita e combina dati globali con la cronologia. `Vulnerabilities.tsx` e `DataLeaks.tsx` aggiungono stato locale per filtri e ricerca. `Network.tsx` traduce dodici porte note in servizi leggibili e ordina le esposizioni. `CertsEmail.tsx` interpreta come spoofabile ogni stringa contenente "possible"; nel seed "Spoofing possible." viene quindi correttamente segnalata come critica. `SimilarDomains.tsx` filtra le varianti sintetiche per nome. L'insieme realizza una navigazione tematica senza duplicare il caricamento del riepilogo.

Backend Python e API

Logica condivisa in `api/_data.py`

Il modulo `_data.py` è il nucleo applicativo del backend. Usa `pathlib` per individuare il dataset, `json` per caricarlo, `random` per la generazione pseudo-casuale, `datetime` per le date e `hashlib.sha1` per identificativi ripetibili. Il generatore `_rng` inizializza un'istanza separata con la stringa composta da dominio e "salt"; per questo la sequenza non varia fra richieste a parità di runtime Python e codice.

I profili sintetici hanno un valore guida denominato `seed`, pari a 47 per Northwind e 12 per Aurora. Tale valore è usato come punteggio complessivo e come parametro centrale di diverse distribuzioni gaussiane. La funzione `_scaled_score` estrae un valore con deviazione standard 12 e lo limita fra zero e cento. Asset, porte, certificati, vulnerabilità e leak applicano formule diverse, descritte nel capitolo dedicato alla provenienza dei dati.

FastAPI per lo sviluppo locale

`server/app/main.py` aggiunge la cartella `api` al percorso di importazione e crea un'app FastAPI denominata "Perimetra Security Report API", versione 1.0.0. Il middleware CORS accetta ogni origine, metodo e header. Le route dichiarano parametri `Query`, restituiscono 404 per un dominio ignoto e, nel caso di `days`, applicano convalida fra 7 e 365. FastAPI genera automaticamente OpenAPI, Swagger UI su `/docs` e ReDoc su `/redoc`. Uvicorn svolge il ruolo di server ASGI locale.

Funzioni serverless per Vercel

`api/_util.py` definisce `JSONHandler` sopra `BaseHTTPRequestHandler`. Il metodo `_query` analizza i parametri con `urllib.parse`; `_send` serializza la risposta UTF-8, imposta il content type e gli header CORS; `do_OPTIONS` risponde al preflight; `do_GET` intercetta eccezioni generiche e restituisce HTTP 500. I file `domains.py`, `summary.py`, `history.py`, `vulnerabilities.py`, `dataleaks.py` e `similar.py` adattano ciascuna funzione dati a una route Vercel.

FastAPI e serverless non sono perfettamente equivalenti nella validazione. FastAPI restituisce HTTP 422 per `days` non numerico o fuori intervallo; la funzione serverless converte esplicitamente a intero, limita i numeri validi a 7–365 e produce HTTP 500 se la conversione fallisce. Inoltre `/api/health` esiste soltanto nel server locale. Queste differenze sono verificabili nel codice e dovrebbero essere uniformate in un'evoluzione produttiva.

Contratto degli endpoint

`GET /api/domains` non richiede parametri e restituisce per ciascun dominio nome, rischio convertito a intero, etichetta e ultima modifica. `GET /api/summary` accetta il dominio opzionale e restituisce il report dentro un array `results`. `GET /api/history` accetta dominio e numero di giorni, con default 90. `GET /api/vulnerabilities`, `/api/dataleaks` e `/api/similar` restituiscono i rispettivi dettagli sintetici. In assenza del parametro dominio viene sempre adottato `cybersonar.demo`.

Le risposte riuscite includono `status: "success"`. Per un dominio sconosciuto FastAPI usa il campo `detail` standard delle eccezioni HTTP, mentre gli handler serverless usano `status: "error"` e `message`. Il client frontend cerca soprattutto `message`; di conseguenza un 404 FastAPI viene mostrato con il generico "Request failed: 404", mentre in produzione può riportare il messaggio specifico. Anche questa è una discrepanza minore ma reale.

Tecnologie, framework e dipendenze

Dipendenze runtime del frontend

React, dichiarato come `^18.3.1`, fornisce componenti, hook ed elaborazione dichiarativa dell'interfaccia. **React DOM**, anch'esso `^18.3.1`, collega React al documento del browser mediante `createRoot`. **React Router DOM**, dichiarato `^6.26.0`, implementa routing, route, link e navigazione attiva. Nel lockfile e nell'installazione analizzata le versioni risolte risultano rispettivamente React 18.3.1, React DOM 18.3.1 e React Router DOM 6.30.4, compatibili con gli intervalli dichiarati.

Non risultano dipendenze per grafici, stato globale, CSS-in-JS, richieste HTTP o validazione. Grafici e gauge sono implementati con SVG e CSS; il Context nativo di React gestisce lo stato condiviso; `fetch` esegue le richieste. Questa scelta riduce le dipendenze dirette, ma attribuisce al codice del progetto la responsabilità di accessibilità, interazione e robustezza delle visualizzazioni.

Strumenti di sviluppo

Vite, dichiarato `^5.4.0` e risolto 5.4.21, fornisce server di sviluppo, trasformazione e build. **@vitejs/plugin-react**, dichiarato `^4.3.1` e risolto 4.7.0, integra la trasformazione JSX/TSX. **TypeScript**, dichiarato `^5.5.4` e risolto 5.9.3, esegue il type-check in modalità strict senza emettere file. **@types/react** e **@types/react-dom** forniscono le dichiarazioni di tipo necessarie all'editor e al compilatore; non vengono inclusi nel bundle runtime.

La configurazione TypeScript mira a ES2020, usa risoluzione "bundler", JSX automatico, moduli ESNext, controlli strict e alias `@/*`. Vite replica l'alias tramite il modulo standard Node `path`, serve sulla porta 5173 e inoltra `/api` alla porta 8000. Lo script di build esegue prima `tsc --noEmit` e poi `vite build`. La verifica effettuata ha trasformato 54 moduli e prodotto il documento HTML, il foglio CSS e il bundle JavaScript ottimizzato.

Dipendenza dichiarata in modo incompleto. `package.json` contiene lo script `"lint": "eslint ."`, ma ESLint non compare fra le dipendenze e non esiste una configurazione nel repository. Lo script non può quindi essere considerato operativo nello stato corrente. Non sono presenti neppure uno script `test` o framework di test automatici.

Python e librerie backend

`server/requirements.txt` dichiara **FastAPI** almeno alla versione 0.110 e **Uvicorn** con extra `standard` almeno alla versione 0.29. FastAPI definisce route, query, CORS, errori HTTP e documentazione OpenAPI; Uvicorn avvia l'app ASGI durante lo sviluppo. Le funzioni Vercel non importano questi pacchetti: utilizzano esclusivamente la libreria standard Python. Il runtime di produzione configurato è Python 3.12.

I moduli standard hanno ruoli specifici. `json` legge il seed e serializza le risposte; `random` genera valori demo; `hashlib` produce identificativi sintetici; `datetime` e `timedelta` costruiscono date e serie storiche; `pathlib` risolve i percorsi; `sys` consente al server locale di importare la logica condivisa; `http.server` e `urllib.parse` implementano gli handler serverless. `from __future__ import annotations` rende differita la valutazione delle annotazioni nei moduli che lo dichiarano.

API native del browser e CSS

Il progetto usa `localStorage` per le preferenze, `matchMedia` per il tema di sistema, `ResizeObserver` per la responsività del grafico, `Intl.NumberFormat` per i numeri, `encodeURIComponent` per il dominio nelle query e SVG per le visualizzazioni. HTML5 e CSS nativo completano lo stack. Le dipendenze transitive del lockfile, fra cui strumenti interni richiesti da Vite, React Router, Babel ed esbuild, non rappresentano scelte applicative dirette e non sono importate dal codice di Perimtra.

Il JSON di riferimento e il modello informativo

Struttura generale

`data/summary_seed.json` contiene un oggetto radice con `status: "success"` e un array `results` composto da un solo report. Il modulo Python legge esclusivamente `results[0]`; il valore radice `status` non viene conservato nel modello interno, perché ogni adattatore HTTP crea autonomamente lo stato della risposta. Il report appartiene a `cybersonar.demo` ed è datato 7 marzo 2024, con ultima modifica l'8 marzo 2024.

Identità, testi e punteggi

`idsummary` identifica il report; `domain_name` specifica il dominio; `creation_date` e `last_edit` ne descrivono la cronologia amministrativa. `summary_text` e `summary_text_en` contengono riepiloghi in italiano e inglese con una sintassi parzialmente Markdown. `risk_score` è memorizzato come stringa, non come numero, e vale "99". Il frontend lo converte quando deve applicare soglie o disegnare il gauge.

I punteggi specifici sono numeri da interpretare nel contesto del report fornito, non formule calcolate da Perimetra. `servizi_esposti_score` vale 99, `dataleak_score` 100, `rapporto_leak_email_score` 50, `spoofing_score` 50, `open_ports_score` 1, `blacklist_score` 70, `vulnerability_score_active` 54, `vulnerability_score_passive` 99 e `certificate_score` 61. L'applicazione li visualizza, ma non possiede la metodologia originaria che li ha prodotti. Non è quindi corretto dedurre dal codice, per esempio, come il punteggio porte aperte pari a 1 sia stato calcolato.

Rete, certificati e protezioni

`n_asset` indica 102 asset; `unique_ipv4` e `unique_ipv6` indicano rispettivamente 30 e 23 indirizzi unici. `n_port` associa ogni porta a un oggetto con campo `n`: il seed riporta 3 esposizioni sulla porta 53, 68 sulla 80, 42 sulla 443, 9 sulla 6667, 9 sulla 6697, 6 sulla 8080 e 21 sulla 8800. Questi conteggi rappresentano esposizioni e non necessariamente porte distinte su un singolo host.

`n_cert_attivi` vale 15 e `n_cert_scaduti` 18. L'oggetto `waf` dichiara quattro asset protetti e ne conserva gli identificativi; `cdn` dichiara conteggio zero e array vuoto. Il frontend ricava testi quali "Protezione attiva" o "Nessuna CDN — rischio DDoS" confrontando questi conteggi con zero. Quest'ultima frase è un'indicazione visuale prudenziale del frontend, non una misurazione di attacchi DDoS.

Sicurezza della posta

`email_security.spoofable` contiene la frase "Spoofing possible." e viene interpretata con una ricerca testuale case-insensitive della parola "possible". `dmARC_policy` vale `quarantine`. Le rilevazioni in blacklist sono zero su sessanta liste e l'elenco delle liste positive è vuoto. Il frontend considera `reject`

la configurazione DMARC più protettiva e, quando la policy è diversa, mostra un suggerimento. Il progetto non esegue verifiche DNS di SPF o DMARC: presenta esclusivamente il valore del dataset.

Vulnerabilità aggregate

`n_vulns` separa totale, attive e passive. Nel seed compaiono 2 vulnerabilità critiche, 18 alte, 52 medie, nessuna bassa e 90 informative, per un totale derivato di 162. Le attive sono 2 critiche, 3 medie e 90 informative, quindi 95 complessive; le passive sono 18 alte e 49 medie, quindi 67. In ciascuna gravità la somma attive più passive coincide con il totale. Il JSON non contiene titoli, CVE, asset o date per le singole vulnerabilità: tali righe vengono create successivamente dal backend.

Data leak aggregati

`n_dataleak` usa cinque categorie. I totali del seed sono zero VIP, un `domain_stealer`, 826 `potential_stealer`, 11 `other_stealer` e zero `general_leak`, per una somma derivata di 838. Tutti risultano non risolti. `enumeration` vale 2, ma non viene visualizzato direttamente dalle pagine analizzate. Il JSON non include fonti, indirizzi email o date delle singole fughe: anche questi dettagli sono simulati.

Domini simili e riepilogo testuale

`n_similar_domains` vale 13, ma il seed non contiene l'elenco dei domini. Il riepilogo testuale descrive in prosa i valori aggregati e viene mostrato quasi integralmente nella panoramica. Alcune imprecisioni ortografiche e tipografiche presenti nel testo originario vengono conservate, salvo la rimozione dei marcatori `**`. Perimetra non rigenera il riepilogo di `cyberpersonar.demo` a partire dai numeri: utilizza il testo fornito.

Provenienza dei dati: originali, derivati, sintetici e predefiniti

Dati originali conservati

Per `cybersonar.demo` il backend conserva tutti i campi del primo risultato JSON: identificativo, testi, punteggi, date, porte, asset, indirizzi IP, certificati, conteggio di domini simili, sicurezza email, aggregati dei leak, aggregati delle vulnerabilità, WAF e CDN. La funzione `_build_reports` crea una copia superficiale del dizionario e aggiunge soltanto `label: "Critico"`. I dati nidificati non vengono mutati dal flusso corrente.

La presenza dei numeri nel seed non dimostra tuttavia che essi provengano da una scansione reale. Il brief li presenta come risultato di una chiamata API e il dominio è esplicitamente dimostrativo. La distinzione corretta è quindi fra “dato originale del file fornito” e “dato sintetizzato dall’applicazione”, non fra “misurazione reale” e “dato falso”. Tutto il progetto è una demo e nessuna provenienza esterna è verificabile dal repository.

Valori derivati nel frontend

Il totale delle vulnerabilità è la somma di critical, high, medium, low e info; il totale dei leak è la somma delle cinque categorie. Le barre impilate calcolano per ogni segmento $\text{valore} / \text{somma} \times 100$; le barre orizzontali usano $\text{valore} / \text{massimo} \times 100$. Il gauge trasforma il rischio in una porzione di un arco pari al 75% della circonferenza. I meter calcolano $\text{valore} / \text{massimo} \times 100$ e limitano il risultato all’intervallo visuale.

Le etichette di rischio derivano dalle soglie 25, 50 e 75. I nomi dei servizi derivano da una mappa statica delle porte; una porta non mappata viene definita “sconosciuto”. “Spoofing possibile” deriva dalla presenza di “possible” nella stringa del report; la bontà DMARC deriva dall’uguaglianza con `reject`. I conteggi risolti e non risolti della pagina leak sono somme delle categorie. Le percentuali di similarità sono il valore sintetico moltiplicato per cento e arrotondato.

Report interamente sintetici

`northwind-retail.demo` e `aurora-fintech.demo` non sono presenti nel JSON. Il codice assegna loro rischi guida 47 e 12, con etichette “Attenzione” e “Solido”. Per ciascun dominio crea asset con una gaussiana centrata circa sul 90% del rischio, ricava IPv4 come fra il 25% e il 45% degli asset e IPv6 fra il 5% e il 25%, con limiti minimi. Seleziona da tre a sette porte comuni e genera esposizioni gaussiane, attribuendo peso 1,6 alle porte 80 e 443.

I certificati attivi sono centrati su $20 - \text{rischio} \times 0,1$, quelli scaduti su $\text{rischio} \times 0,15$. Le vulnerabilità totali hanno medie proporzionali al rischio: 3% per le critiche, 18% per le alte, 50% per le medie, 10% per le basse e 70% per le informative, con deviazioni specifiche. Una frazione casuale fra 5% e 35% viene marcata attiva; il resto diventa passivo. I leak usano analogamente gaussiane per categoria, con `potential_stealer` centrato su sette volte il rischio; una frazione fra zero e 60% viene marcata risolta.

La possibilità di spoofing è estratta con probabilità pari al rischio percentuale. Se il rischio supera 30, la policy DMARC è scelta fra `none`, `quarantine` e `reject`; altrimenti viene impostata a `reject`. Il conteggio WAF deriva da una gaussiana centrata su $4 - \text{rischio} \times 0,02$; la CDN può valere zero o uno. Le date partono dal riferimento fisso 7 marzo 2024 e sono arretrate da uno a quaranta giorni. I punteggi specifici sono gaussiane limitate fra zero e cento, con centri proporzionali al rischio; lo spoofing score è 50 oppure 5.

Con l'algoritmo e l'ambiente analizzati, Northwind produce 49 asset, 12 IPv4, 8 IPv6, 56 vulnerabilità aggregate e 358 leak aggregati; Aurora produce 23 asset, 6 IPv4, 4 IPv6, 7 vulnerabilità e 176 leak. Questi numeri sono ripetibili, ma restano simulazioni: la ripetibilità non li rende osservazioni reali.

Dettagli sintetici costruiti sopra gli aggregati

La cronologia è inventata per tutti e tre i domini, incluso `cybersonar.demo`. Il backend sceglie un punteggio iniziale spostato fra -25 e +15 rispetto al rischio finale, interpola linearmente verso il valore corrente, aggiunge rumore gaussiano con deviazione 2,5 e forza l'ultimo punto a coincidere con il report. Con novanta giorni, Cybersonar va dal valore sintetico 78 del 10 dicembre 2023 al valore originale 99 dell'8 marzo 2024. Non si tratta di misure raccolte nel tempo.

Le righe delle vulnerabilità rispettano per quantità, gravità e stato gli aggregati del report, ma identificativi, titoli, asset e date sono inventati. I titoli vengono scelti da piccoli cataloghi statici, gli asset hanno forma `asset-N.dominio` e le date cadono entro sessanta giorni dall'ultima modifica. Per Cybersonar vengono generate 162 righe, ma nessuna corrisponde a un CVE realmente attestato dal JSON.

I dettagli dei leak usano sorgenti scelte da cinque descrizioni predefinite, identificativi email `userNNN@dominio` e date entro centoventi giorni. Per evitare output eccessivi, sono create al massimo venticinque righe per categoria: Cybersonar mostra quindi 37 record campione a fronte di 838 eventi aggregati. La logica assegna lo stato "resolved" alle prime righe di ciascuna categoria fino al conteggio risolto, ma se i risolti superano il campione il dettaglio non rappresenta proporzionalmente l'intero insieme.

I domini simili sono costruiti combinando prefissi e suffissi quali `secure-`, `login-`, `-support` o `-verify` con TLD scelti da un insieme statico. Similarità, registrazione e rischio sono casuali deterministici; il codice tenta di evitare duplicati. Il numero desiderato proviene da `n_similar_domains`, ma nomi e valutazioni non derivano da DNS, WHOIS o threat intelligence.

Valori predefiniti e fallback

Quando una query omette il dominio, backend e frontend adottano `cybersonar.demo`. La cronologia usa 90 giorni se il chiamante non specifica `days`. Il tema deriva dal sistema se non è salvato; il dominio deriva dal default se non è salvato. Il grafico orizzontale usa almeno 1 come massimo per evitare divisioni problematiche; la barra impilata usa totale 1 quando tutti i segmenti sono zero. I messaggi di errore hanno testi generici se l'eccezione non espone un messaggio utile.

Regola di interpretazione. Nessun dettaglio generato deve essere presentato come evidenza di una scansione. L'applicazione dimostra la forma di un possibile prodotto e il comportamento dell'interfaccia, non l'effettiva postura di sicurezza di un'organizzazione.

Installazione, avvio e utilizzo

Prerequisiti

Il repository non dichiara un campo `engines`. Il codice Python richiede almeno Python 3.10 per la sintassi di unione `dict | None`; Python 3.12 è la scelta coerente con Vercel. Vite 5 richiede un ambiente Node moderno; Node.js 18 o successivo è una base appropriata. Sono inoltre necessari npm e, per ottenere il progetto dal repository, Git. Non sono richiesti database, chiavi API o variabili d'ambiente.

Clonazione e backend

Il repository ufficiale può essere clonato e aperto con i comandi seguenti:

```
git clone https://github.com/Emilio-Moscatiello/Perimetra-brief-.git
cd Perimetra-brief-
```

In PowerShell, dalla radice del progetto, è opportuno creare un ambiente virtuale, attivarlo e installare le dipendenze locali:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r server\requirements.txt
```

Il backend viene avviato dalla radice. L'opzione reload è adatta allo sviluppo:

```
python -m uvicorn server.app.main:app --reload --port 8000
```

Il controllo `http://127.0.0.1:8000/api/health` deve restituire `{"status":"ok"}`. La documentazione interattiva è disponibile su `/docs` e `/redoc`.

Frontend

In un secondo terminale si installano le dipendenze bloccate dal lockfile e si avvia Vite:

```
cd frontend
npm install
npm run dev
```

L'interfaccia è raggiungibile su <http://localhost:5173>. Il proxy configurato da Vite inoltra tutte le richieste `/api` a FastAPI, evitando di inserire URL assolute nel frontend. L'utente può selezionare il dominio nella barra superiore, navigare fra le sezioni, applicare filtri e passare dal tema chiaro a quello scuro. Poiché le preferenze sono locali al browser, cancellare il `localStorage` ripristina i default.

Build e anteprima di produzione

Dalla cartella `frontend`, `npm run build` esegue il controllo TypeScript e genera `dist`. La build è stata verificata con successo durante la redazione di questa documentazione. `npm run preview` serve localmente gli artefatti già prodotti, ma non avvia le API Python: per una verifica completa occorre mantenere disponibile un backend oppure usare l'ambiente Vercel.

Distribuzione su Vercel

Configurazione di build

`vercel.json` esegue `cd frontend && npm install && npm run build` e pubblica `frontend/dist`. Le funzioni corrispondenti a `api/*.py` usano il runtime Python 3.12 e includono `data/**`, condizione necessaria perché `_data.py` possa aprire il seed. Un `rewrite` invia ogni percorso che non inizia con `api/` a `/index.html`, consentendo al `BrowserRouter` di gestire direttamente le route interne.

Procedura e comportamento in produzione

Il progetto può essere importato in Vercel collegando il repository GitHub. La configurazione risiede nella radice, quindi non è necessario impostare manualmente una root directory differente. Non risultano variabili d'ambiente richieste. Durante il deploy il frontend viene compilato e ogni handler Python diventa una funzione serverless. FastAPI e Uvicorn non vengono avviati in produzione; rimangono strumenti della modalità locale.

Il repository non contiene, né il prompt fornisce, un URL pubblico verificato del deployment. La documentazione non ne inventa uno. Il collegamento certo e ufficiale è il repository GitHub. Prima di presentare un'istanza pubblica come operativa è necessario verificare il deploy, le route API, il caricamento diretto delle route SPA e la presenza del dataset nel bundle serverless.

Limiti, sicurezza e qualità del software

Limiti funzionali

Perimetra non acquisisce dati da domini reali, non interroga DNS, certificati, blacklist o database di vulnerabilità e non effettua remediation. Non esiste persistenza lato server: i report sono ricostruiti in memoria all'avvio. La cronologia non è uno storico, i dettagli non sono evidenze e il selettore è limitato ai tre profili codificati. Non sono implementati autenticazione, autorizzazione, account, ruoli, esportazione, paginazione server-side o confronto fra scansioni.

Considerazioni di sicurezza

Il middleware FastAPI e gli handler serverless accettano richieste CORS da qualsiasi origine. Per una demo pubblica di sola lettura è una configurazione semplice, ma un sistema reale dovrebbe limitare le origini e valutare autenticazione, rate limiting, logging, protezione dagli abusi e gestione centralizzata degli errori. Le API accettano il dominio soltanto come chiave di un dizionario noto, quindi non lo usano per aprire URL o eseguire comandi. L'uso di `encodeURIComponent` evita che il frontend costruisca query ambigue.

Gli identificativi sintetici dei leak assomigliano a indirizzi email, ma sono generati e appartengono a domini `.demo`. In un'applicazione reale, dati personali e credenziali richiederebbero minimizzazione, controllo degli accessi, cifratura, conservazione limitata e tracciamento delle consultazioni. Il progetto non implementa tali misure perché non tratta un dataset operativo.

Qualità, test e manutenzione

La compilazione TypeScript in modalità strict e la build verificata offrono un controllo statico utile. La separazione dei tipi, il riuso dei componenti e l'unificazione della logica dati riducono duplicazioni. Restano però assenti test unitari, di integrazione ed end-to-end; lo script lint non è configurato; non è presente una pipeline CI. Il repository non specifica una licenza, quindi non si devono presumere diritti di riuso oltre a quelli concessi dal proprietario.

Esistono inoltre piccole asimmetrie da gestire: risposta d'errore diversa fra FastAPI e serverless, validazione differente di `days`, endpoint health solo locale, funzione `refresh` non utilizzata, assenza di route 404 e assenza di validazione runtime dei payload. Il nome del pacchetto npm e le chiavi di `localStorage` conservano il precedente identificatore tecnico "cybersonar"; ciò non cambia il brand visibile, ma può essere uniformato.

Accessibilità e compatibilità

Il progetto include focus visibili, media query, riduzione del movimento, etichette ARIA e contrasto tematico progettato per stato. Tuttavia non risultano misurazioni automatiche del contrasto, test di tastiera completi o audit WCAG. L'uso di `color-mix`, `ResizeObserver` e altre API moderne suggerisce di verificare i browser di destinazione. Le tabelle sono scrollabili orizzontalmente su schermi stretti, ma dataset molto grandi richiederebbero virtualizzazione o paginazione.

Sviluppi futuri

Il passaggio da prototipo a prodotto richiederebbe innanzitutto una sorgente dati reale e documentata. Un connettore di assessment dovrebbe separare raccolta, normalizzazione e presentazione, conservare il timestamp e la provenienza di ogni evidenza e rendere trasparente la metodologia dei punteggi. Un database permetterebbe di sostituire la cronologia simulata con scansioni persistenti, confrontare versioni del report e supportare più organizzazioni.

Sul backend sarebbero opportuni schemi di risposta condivisi, validazione uniforme, gestione degli errori coerente, autenticazione e autorizzazione per ruoli, rate limiting, log strutturati e test di contratto fra FastAPI e funzioni serverless. Sul frontend si potrebbero aggiungere paginazione, ordinamento, viste comparate, esportazione PDF o CSV, gestione di dataset estesi e una route di errore. Test unitari per formule e filtri, test di integrazione per le API e test end-to-end per la navigazione ridurrebbero il rischio di regressioni.

La qualità del repository migliorerebbe aggiungendo ESLint con configurazione coerente a TypeScript e React, una pipeline CI che esegua type-check, test e build, un file di licenza, versioni minime dei runtime e documentazione del deployment pubblico. Un audit accessibilità e una revisione di sicurezza dovrebbero precedere qualunque impiego oltre la dimostrazione.

CONCLUSIONI

Valutazione complessiva e riferimenti

Perimetra soddisfa il nucleo del brief trasformando un report JSON complesso in un'esperienza full stack leggibile e navigabile. Il frontend è compatto, tipizzato e privo di dipendenze grafiche superflue; il backend condivide la logica fra ambiente locale e serverless; il dataset viene ampliato con simulazioni deterministiche capaci di mostrare grafici, filtri e sezioni specialistiche. La build di produzione verificata conferma la coerenza tecnica del frontend nello stato analizzato.

Il valore del progetto dipende anche dalla chiarezza con cui se ne dichiarano i confini. Soltanto il riepilogo di [cybersonar.demo](#) è letto dal JSON fornito, e anche quel file ha natura dimostrativa. Gli altri domini, la cronologia e tutte le evidenze granulari sono inventati dal codice. La dashboard rappresenta quindi un prototipo credibile di consultazione, non uno strumento di security assessment operativo.

Il repository ufficiale e unico riferimento versionato è <https://github.com/Emilio-Moscatiello/Perimetra-brief>. Per clonarlo si può utilizzare l'URL Git <https://github.com/Emilio-Moscatiello/Perimetra-brief.git>.

Documento redatto in lingua italiana sulla base del contenuto del repository locale. Data di verifica: 13 luglio 2026. Le funzionalità future descritte sono proposte e non devono essere interpretate come già implementate.